

Phase 2: Insertion and In-Place Heap Creation

1. Current Progress

You have already completed **Phase 1: Binary Heap Foundations**.

That means you should already understand these ideas:

- A heap is a **complete binary tree**.
- A heap is usually stored inside an **array**.
- In this curriculum, **index 0 is ignored**.
- The root of the heap is at **index 1**.
- For a node at index `i`:
- Left child = `2 * i`
- Right child = `2 * i + 1`
- Parent = `i / 2`
- A **max heap** keeps the largest value at the root.
- A **min heap** keeps the smallest value at the root.
- A heap is **not fully sorted**.
- A heap only guarantees the correct **parent-child relationship**.

Now Phase 2 teaches you how to **insert new values into a heap** and how to **create a heap from an unsorted array**.

2. What This Phase Covers

In this phase, you will learn:

1. The main rule of heap insertion.
2. Why insertion always happens at the next free position.
3. How insertion preserves the complete binary tree shape.
4. What **shift-up** means.
5. What **percolate-up** means.
6. What **bubble-up** means.
7. Why these three words mean almost the same thing.
8. How to insert into a **max heap**.
9. Why we use a temporary variable called `temp`.
10. The C function for heap insertion.
11. Dry run for inserting `40`.
12. Dry run for inserting `35`.
13. How to create a heap using repeated insertion.
14. What **in-place heap creation** means.
15. What the **heap side** and **unsorted side** mean.
16. The `createHeap` function.

17. Dry run for creating a heap from `[10, 20, 30, 25, 5, 40, 35]`.
 18. Time complexity of insertion.
 19. Time complexity of heap creation by repeated insertion.
 20. Space complexity.
 21. How max heap insertion changes into min heap insertion.
 22. Common beginner mistakes.
-

3. Big Idea of Phase 2

In Phase 1, you learned what a heap **is**.

In Phase 2, you learn how to **modify** a heap.

The first modifying operation is:

Insertion

Insertion means:

Adding a new value into the heap.

But we cannot add the value anywhere we like.

A heap must always keep two rules:

1. Complete binary tree shape
2. Heap ordering rule

For a max heap, the ordering rule is:

Parent $\geq$ Child

For a min heap, the ordering rule is:

Parent $\leq$ Child

In this phase, we mainly focus on **max heap insertion**.

4. The Main Rule of Heap Insertion

The most important rule is:

Always insert the new value at the next free position.

This means the new value is first placed at the end of the heap array.

Why?

Because this keeps the heap as a **complete binary tree**.

Remember:

A complete binary tree is filled level by level, from left to right.

So when we insert a new value, we must place it in the next available position from left to right.

5. Example: Insert at the Next Free Position

Suppose this is our max heap:

Index:	1	2	3	4	5	6	7
Value:	40	30	35	10	20	25	15

Tree view:

```
      40
     /
    30  35
   / \  /
  10 20 25 15
```

This is a complete binary tree.

If we insert a new value, the next free position is index **8**.

So the new value must first go here:

Index:	1	2	3	4	5	6	7	8
Value:	40	30	35	10	20	25	15	new

This keeps the shape correct.

6. Why the New Value May Need to Move Up

After placing the new value at the end, the shape is correct.

But the heap order may be wrong.

Example:

Suppose we insert 50.

First place 50 at index 8:

Index:	1	2	3	4	5	6	7	8
Value:	40	30	35	10	20	25	15	50

Now find the parent of index 8:

Parent index = $8 / 2 = 4$

So the parent is:

$A[4] = 10$

Now compare:

$50 > 10$

This breaks the max heap rule because the child is bigger than the parent.

In a max heap, the parent should be greater than or equal to the child.

So 50 must move upward.

7. Shift-Up, Percolate-Up, and Bubble-Up

When the inserted value moves upward, this is called:

Shift-up

It can also be called:

Percolate-up

Or:

Bubble-up

These names mean almost the same thing.

Simple meaning:

The new value climbs upward until it reaches the correct position.

For a max heap:

A larger value moves upward.

For a min heap:

A smaller value moves upward.

8. Max Heap Insertion Logic

For max heap insertion:

If the new value is bigger than its parent, move it upward.

But in the actual code, we do not keep swapping every time.

Instead, we use a smarter method:

1. Store the new value in temp.
2. Move smaller parents downward.
3. Finally place temp in the correct position.

This is efficient because the new value is saved safely and we only place it once at the end.

9. Why We Use temp

Suppose we insert a value and it needs to move upward.

If we directly overwrite array positions, we may lose the inserted value.

So we first save it:

```
int temp = A[n];
```

Here:

- A is the heap array.
- n is the index where the new value was inserted.
- temp stores the inserted value safely.

Simple meaning:

```
temp is like holding the new value in your hand while moving other values down.
```

10. Heap Insertion Algorithm in Simple Steps

Assume the new value is already placed at index n.

For max heap insertion:

1. Store A[n] in temp.
2. Set i = n.
3. Compare temp with its parent A[i / 2].
4. If temp is greater than the parent, move the parent down.
5. Move i upward to the parent index.

6. Repeat until temp is no longer greater than the parent.
7. Put temp in the final correct position.

The loop stops when:

1. The value reaches the root.

or:

2. The parent is already greater than or equal to temp.

11. Insert Function for a Max Heap

```
void insert(int A[], int n) {  
    int temp = A[n];  
    int i = n;  
  
    while (i > 1 && temp > A[i / 2]) {  
        A[i] = A[i / 2];  
        i = i / 2;  
    }  
  
    A[i] = temp;  
}
```

12. Line-by-Line Explanation of `insert`

Line 1

```
void insert(int A[], int n)
```

This function inserts the value at index `n` into the correct heap position.

Important:

```
The value is already placed at A[n] before calling this function.
```

Line 2

```
int temp = A[n];
```

This stores the new value safely.

Why?

Because during shift-up, parent values may be copied downward into the array.

Without `temp`, the new value could be overwritten.

Line 3

```
int i = n;
```

This means we start checking from the inserted value's index.

So `i` represents the current empty/candidate position.

Line 4

```
while (i > 1 && temp > A[i / 2])
```

This loop continues while both conditions are true.

Condition 1:

```
i > 1
```

This means we are not at the root yet.

Condition 2:

```
temp > A[i / 2]
```

This means the inserted value is greater than its parent.

If both are true, the inserted value must move upward.

Line 5

```
A[i] = A[i / 2];
```

This moves the parent value down.

Simple meaning:

```
The parent is smaller than temp, so it must go one level lower.
```

Line 6

```
i = i / 2;
```

This moves `i` upward to the parent index.

Now we check the next parent above.

Line 7

```
A[i] = temp;
```

When the loop stops, we place the inserted value into its final correct position.

13. Dry Run 1: Inserting 40

Suppose we insert `40` at index `8`.

The path upward is:

```
Index 8 -> Index 4 -> Index 2 -> Index 1
```

Suppose the values on that path are:

Index 8 = 40
Index 4 = 5
Index 2 = 20
Index 1 = 30

Now we apply shift-up.

Step 1

Compare 40 with its parent 5:

40 > 5

So move 5 down to index 8.

Now the empty position moves up to index 4.

Step 2

Compare 40 with parent 20:

40 > 20

So move 20 down to index 4.

Now the empty position moves up to index 2.

Step 3

Compare 40 with parent 30:

40 > 30

So move 30 down to index 2.

Now the empty position moves up to index 1.

Step 4

Now `i == 1`.

That means we reached the root.

Place `40` at index `1`.

Final result:

40 becomes the root.

Reason:

40 was larger than every ancestor on its path.

14. Dry Run 2: Inserting 35

Suppose we insert `35`.

Its upward path contains:

20 -> 30 -> 40

Now compare step by step.

Step 1

Compare `35` with parent `20`:

35 > 20

So move `20` down.

`35` continues upward.

Step 2

Compare 35 with parent 30 :

35 > 30

So move 30 down.

35 continues upward.

Step 3

Compare 35 with parent 40 :

35 > 40

This is false.

So stop.

Place 35 below 40 .

Final result:

35 moves upward, but it does not become the root.

Reason:

40 is still larger than 35.

15. Important Direction Rule

Insertion moves:

From bottom to top

or:

From leaf toward root

So during insertion, we compare the inserted value with its:

Parent

Not its children.

This is very important.

Many beginners confuse insertion with deletion.

Remember:

Insertion moves upward.
Deletion moves downward.

Deletion is covered in Phase 3.

16. Why Insertion Takes $O(\log N)$

A heap is a complete binary tree.

A complete binary tree stays compact and balanced.

So its height is:

$O(\log N)$

During insertion, the new value moves only along one path:

leaf -> parent -> parent -> root

It does not check all elements.

It does not search the whole array.

It only moves up through the height of the heap.

Therefore:

Insertion time complexity = $O(\log N)$

17. Creating a Heap from an Unsorted Array

Now we move to the second main topic of Phase 2.

We want to take a normal unsorted array and rearrange it into a heap.

Example:

10, 20, 30, 25, 5, 40, 35

At first, this is not a heap.

Our goal is to rearrange it into a valid max heap.

18. What In-Place Heap Creation Means

In-place means:

We do not use another array.

Instead:

We rearrange the same array.

So if the original array is:

[10, 20, 30, 25, 5, 40, 35]

After heap creation, the same array may become:

[40, 25, 35, 10, 5, 20, 30]

No second array is needed.

19. Heap Side and Unsorted Side

When creating a heap by repeated insertion, imagine the array has two parts:

Heap side | Unsorted side

At the beginning, only the first element is considered the heap side.

Example:

[10 | 20, 30, 25, 5, 40, 35]

The heap side is:

[10]

A single value is already a valid heap.

Then we insert the next value into the heap side.

After inserting 20:

[20, 10 | 30, 25, 5, 40, 35]

Now the heap side has two elements:

[20, 10]

The boundary moves right each time.

20. Heap Creation by Repeated Insertion

The idea is:

1. Treat A[1] as a one-element heap.
2. Insert A[2] into that heap.
3. Insert A[3] into that heap.
4. Continue until A[n].

This gradually grows the heap.

Each new element is inserted using the same `insert` function.

21. `createHeap` Function

```
void createHeap(int A[], int n) {  
    for (int i = 2; i <= n; i++) {  
        insert(A, i);  
    }  
}
```

22. Explanation of `createHeap`

Line 1

```
void createHeap(int A[], int n)
```

This function creates a heap from the array `A`.

Here:

- `A` is the array.
 - `n` is the number of active elements.
-

Line 2

```
for (int i = 2; i <= n; i++)
```

The loop starts at index `2`.

Why not index `1`?

Because:

A[1] alone is already a valid heap.

A one-node tree always satisfies the heap property.

Line 3

```
insert(A, i);
```

This inserts the element at index `i` into the heap side.

Each time this runs, the heap side becomes one element larger.

23. Dry Run: Create Heap from [10, 20, 30, 25, 5, 40, 35]

Original array using 1-based indexing:

Index:	1	2	3	4	5	6	7
Value:	10	20	30	25	5	40	35

Index `0` is ignored.

So in C, the full array is:

```
int A[] = {0, 10, 20, 30, 25, 5, 40, 35};
```

24. Step 1: Start with 10

Heap side:

[10]

A single value is always a valid heap.

No movement is needed.

25. Step 2: Insert 20

Before adjustment:

[10, 20]

Compare 20 with its parent 10:

20 > 10

So 20 moves up.

Heap side becomes:

[20, 10]

Tree view:

```
  20
 /
10
```

This is a valid max heap.

26. Step 3: Insert 30

Before adjustment:

[20, 10, 30]

Compare 30 with its parent 20:

30 > 20

So 30 moves up.

Heap side becomes:

[30, 10, 20]

Tree view:

```
  30
 /
10  20
```

This is a valid max heap.

27. Step 4: Insert 25

Before adjustment:

[30, 10, 20, 25]

25 is at index 4.

Parent index:

$4 / 2 = 2$

Parent value:

$A[2] = 10$

Compare:

$25 > 10$

So move 10 down.

Now 25 moves to index 2.

Compare 25 with parent 30:

25 > 30

False.

So stop.

Heap side becomes:

[30, 25, 20, 10]

Tree view:

```
      30
     /
    25  20
   /
  10
```

This is a valid max heap.

28. Step 5: Insert 5

Before adjustment:

[30, 25, 20, 10, 5]

5 is at index 5.

Parent index:

$5 / 2 = 2$

Parent value:

$A[2] = 25$

Compare:

5 > 25

False.

So 5 stays where it is.

Heap side becomes:

[30, 25, 20, 10, 5]

Tree view:

```
      30
     /
    25  20
   /
  10  5
```

This is a valid max heap.

29. Step 6: Insert 40

Before adjustment:

[30, 25, 20, 10, 5, 40]

40 is at index 6.

Parent index:

$6 / 2 = 3$

Parent value:

$A[3] = 20$

Compare:

40 > 20

So move 20 down.

Now 40 moves to index 3.

Parent index:

$3 / 2 = 1$

Parent value:

A[1] = 30

Compare:

40 > 30

So move 30 down.

Now 40 reaches index 1.

Heap side becomes:

[40, 25, 30, 10, 5, 20]

Tree view:

```
      40
     /
    25  30
   / \  /
  10  5 20
```

This is a valid max heap.

30. Step 7: Insert 35

Before adjustment:

[40, 25, 30, 10, 5, 20, 35]

35 is at index 7.

Parent index:

$$7 / 2 = 3$$

Parent value:

$$A[3] = 30$$

Compare:

$$35 > 30$$

So move 30 down.

Now 35 moves to index 3.

Parent index:

$$3 / 2 = 1$$

Parent value:

$$A[1] = 40$$

Compare:

$$35 > 40$$

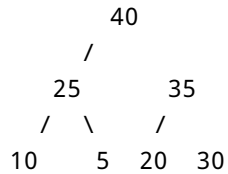
False.

So stop.

Final heap:

[40, 25, 35, 10, 5, 20, 30]

Tree view:



31. Check the Final Heap

Final heap array:

Index:	1	2	3	4	5	6	7
Value:	40	25	35	10	5	20	30

Check parent-child relationships.

Index **1**:

40 >= 25
40 >= 35

Index **2**:

25 >= 10
25 >= 5

Index **3**:

35 >= 20
35 >= 30

All relationships are valid.

So this is a valid max heap.

Important:

The final array is not sorted.

It is a heap.

A heap only needs parent-child order.

32. Complete Phase 2 C Program

```
#include <stdio.h>

void insert(int A[], int n) {
    int temp = A[n];
    int i = n;

    while (i > 1 && temp > A[i / 2]) {
        A[i] = A[i / 2];
        i = i / 2;
    }

    A[i] = temp;
}

void createHeap(int A[], int n) {
    for (int i = 2; i <= n; i++) {
        insert(A, i);
    }
}

void printArray(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

int main() {
    // Index 0 is ignored.
    int A[] = {0, 10, 20, 30, 25, 5, 40, 35};
```

```

int n = 7;

printf("Before heap creation: ");
printArray(A, n);

createHeap(A, n);

printf("After heap creation: ");
printArray(A, n);

return 0;
}

```

33. Expected Output

Before heap creation: 10 20 30 25 5 40 35
 After heap creation: 40 25 35 10 5 20 30

The output after heap creation is:

40 25 35 10 5 20 30

This is not sorted in ascending order.

This is not sorted in descending order.

But it is a valid max heap.

34. Important Variables in the Program

Variable	Meaning
A[]	The heap array
n	Number of active heap elements
temp	Stores the inserted value safely
i	Current position during shift-up
A[i / 2]	Parent of the current index

35. Why the Result Is Not Sorted

A common beginner mistake is expecting this:

```
40 35 30 25 20 10 5
```

But heap creation does not sort the array.

Heap creation only guarantees this:

```
Parent >= Child
```

So this is valid:

```
40 25 35 10 5 20 30
```

Even though 25 comes before 35, the heap is still valid.

Why?

Because 25 and 35 are siblings.

The heap does not require siblings to be sorted.

It only requires parents to be greater than or equal to children.

36. Complexity of One Insertion

Insertion moves upward through the heap.

The longest path is from a leaf to the root.

That path length is the height of the heap.

Because heap height is:

```
 $O(\log N)$ 
```

Insertion is:

$O(\log N)$

37. Complexity of Heap Creation by Repeated Insertion

To create a heap by repeated insertion, we insert many elements.

If there are N elements:

We perform about N insertions.

Each insertion can cost:

$O(\log N)$

So total time is:

$O(N \log N)$

38. Space Complexity

This method is in-place.

That means:

No second array is used.

The program only uses a few variables:

temp, i, loop variables

So extra space is:

$O(1)$

39. Complexity Summary

Operation	Time Complexity	Reason
Insert one value	$O(\log N)$	The value can move up only through the heap height
Create heap by repeated insertion	$O(N \log N)$	There are N insertions, each up to $O(\log N)$
Extra space	$O(1)$	The same array is rearranged

40. How to Change Max Heap Insertion into Min Heap Insertion

For a max heap, bigger values move upward.

So the condition is:

```
temp > A[i / 2]
```

For a min heap, smaller values move upward.

So the condition becomes:

```
temp < A[i / 2]
```

That is the main change.

Max heap:

```
Parent >= Child
```

Min heap:

```
Parent <= Child
```

41. Max Heap vs Min Heap Insertion

Feature	Max Heap	Min Heap
Root contains	Largest value	Smallest value
Parent-child rule	Parent $\geq$ Child	Parent $\leq$ Child
Value that moves up	Bigger value	Smaller value
Main comparison	<code>temp > A[i / 2]</code>	<code>temp < A[i / 2]</code>

42. Common Beginner Mistakes

Mistake 1: Inserting Directly at the Root

Wrong idea:

The new value is large, so I will put it at the root immediately.

Correct idea:

Always insert at the next free position first.
Then shift up if needed.

Why?

Because inserting directly at the root may break the complete binary tree shape.

Mistake 2: Forgetting the Complete Tree Rule

Some learners focus only on values.

But heaps need both:

Shape + Order

So we must first preserve the shape by inserting at the end.

Then we fix the order by shifting up.

Mistake 3: Moving Downward During Insertion

Insertion moves upward.

Wrong direction:

Compare with children and move down.

Correct direction:

Compare with parent and move up.

Remember:

Insertion = upward
Deletion = downward

Mistake 4: Forgetting to Use `temp`

Wrong idea:

I can just keep copying values without saving the inserted value.

Problem:

The inserted value may get overwritten.

Correct idea:

```
int temp = A[n];
```

Save the inserted value first.

Mistake 5: Starting `createHeap` at Index 1

Wrong loop:

```
for (int i = 1; i <= n; i++)
```

Better loop:

```
for (int i = 2; i <= n; i++)
```

Why?

Because:

```
A[1] alone is already a valid heap.
```

Mistake 6: Expecting the Heap to Be Sorted

Wrong expectation:

```
After createHeap, the array should be sorted.
```

Correct idea:

```
After createHeap, the array should satisfy the heap property.
```

A heap is not a sorted array.

It is only partially ordered.

Mistake 7: Forgetting Index 0 Is Ignored

This curriculum uses 1-based indexing.

So the array should look like this:

```
int A[] = {0, 10, 20, 30, 25, 5, 40, 35};
```

The `0` at the beginning is only a dummy value.

The real heap starts at:

A[1]

43. Key Terms

Term	Simple Meaning
Insertion	Adding a new value into the heap
Next free position	The next available array index after the current heap
Shift-up	Moving the inserted value upward
Percolate-up	Another name for shift-up
Bubble-up	Another name for shift-up
<code>temp</code>	A variable that safely stores the inserted value
In-place	Using the same array, not another array
Heap side	The part of the array that already forms a heap
Unsorted side	The part of the array not yet inserted into the heap
Repeated insertion	Creating a heap by inserting elements one by one

44. Important Rules to Memorize

Rule 1

Insertion always starts at the next free position.

Rule 2

Max heap insertion moves larger values upward.

Rule 3

During insertion, compare with the parent.

Rule 4

Use temp to save the inserted value.

Rule 5

A[1] alone is already a heap.

Rule 6

createHeap starts from index 2.

Rule 7

A heap is not fully sorted.

45. Simple Practice Questions

Use this max heap:

Index:	1	2	3	4	5	6	7
Value:	50	30	40	10	20	35	25

Questions

1. Where should a new value be inserted first?
 2. What is the parent index of index 8?
 3. If we insert 60, will it move upward?
 4. If we insert 15, compare it with its parent. Will it move upward?
 5. What direction does insertion move?
 6. What is the time complexity of one insertion?
 7. Does createHeap sort the array?
 8. Why does createHeap start at index 2?
 9. What comparison is used for max heap insertion?
 10. What comparison is used for min heap insertion?
-

46. Practice Answers

1. Where should a new value be inserted first?

At the next free position.

Since the heap has 7 values, the new value goes to index .

Answer:

Index

2. What is the parent index of index .

Formula:

Parent = $i / 2$

So:

$8 / 2 = 4$

Answer:

Index

3. If we insert , will it move upward?

The parent of index is index .

Value at index is .

Compare:

$60 > 10$

Yes, it will move upward.

It may continue moving upward because 60 is very large.

Answer:

Yes

4. If we insert 15, will it move upward?

The parent at index 4 has value 10.

Compare:

15 > 10

Yes, it moves upward once.

Then it compares with index 2, value 30.

15 > 30

False.

So it stops below 30.

Answer:

Yes, but only until it reaches the correct position.

5. What direction does insertion move?

Answer:

Upward, from leaf toward root.

6. What is the time complexity of one insertion?

Answer:

$O(\log N)$

7. Does `createHeap` sort the array?

Answer:

No. It creates a heap, not a sorted array.

8. Why does `createHeap` start at index 2?

Answer:

Because $A[1]$ alone is already a valid heap.

9. What comparison is used for max heap insertion?

Answer:

$\text{temp} > A[i / 2]$

10. What comparison is used for min heap insertion?

Answer:

$\text{temp} < A[i / 2]$

47. Phase 2 Final Summary

Phase 2 teaches how to insert values into a heap and how to create a heap from an unsorted array.

The most important idea is:

Insert at the end first, then shift up.

Insertion must protect the complete binary tree shape.

That is why the new value first goes to the next free position.

After that, insertion fixes the heap order.

For a max heap:

If the inserted value is bigger than its parent, it moves upward.

The inserted value keeps moving upward until:

1. It reaches the root.

or:

2. Its parent is already bigger.

The insert function uses `temp` to safely store the inserted value while parents move downward.

Heap creation by repeated insertion means:

Start with $A[1]$ as a one-node heap.
Then insert $A[2]$, $A[3]$, $A[4]$, ... until $A[n]$.

The method is in-place because it rearranges the original array and does not use another array.

Final complexity:

One insertion = $O(\log N)$
Create heap by repeated insertion = $O(N \log N)$
Extra space = $O(1)$

Most important beginner memory:

Insertion moves upward.
Deletion moves downward.

You are now ready for Phase 3: Deletion and Heap Sort.